

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity. (BL2, BL3)*

Activity Tool : Experiments (High)

Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	5
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	5
3	Given relation R(A,B,C,D,E) with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	5
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	5
5	Verify whether R(A,B,C,D) with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	5
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	5

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	5
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	5
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	5
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	5

Code of Conduct:

I VENNA MEENAKSHI REDDY (192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: VENNA MEENAKSHI REDDY

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation
---------------	---	--	--	----------------------------------

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

Given Relation

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Where:

- **SID** – Student ID
- **SName** – Student Name
- **CID** – Course ID
- **CName** – Course Name
- **Instructor** – Course Instructor
- **Grade** – Grade obtained by the student

1. Understanding of the Relation

The relation stores information about students enrolling in courses.

A student can register for multiple courses, and a course can have multiple students. Therefore, neither SID nor CID alone can uniquely identify a particular record.

For example:

STUDENT_COURSE

SID	SName	CID	CName	Instructor	Grade
101	Arun	201	DBMS	Kumar	A
101	Arun	202	Java	Ravi	B
102	Priya	201	DBMS	Kumar	A
103	Rahul	203	Python	Meena	B

From the above table:

- SID = 101 occurs more than once because Arun takes multiple courses.
- CID = 201 occurs more than once because DBMS has multiple students.
- The combination (SID, CID) uniquely identifies one student's enrollment in one course.

2. Identifying Functional Dependencies

A functional dependency means that one attribute or a set of attributes determines another attribute.

FD 1: Student ID determines Student Name

Every student has one student name.

SID → SName

For example:

101 → Arun

102 → Priya

103 → Rahul

Therefore:

SID → SName

FD 2: Course ID determines Course Name

Each course ID identifies one particular course.

CID → CName

For example:

201 → DBMS

202 → Java

203 → Python

Therefore:

CID → CName

FD 3: Course ID determines Instructor

Assuming each course has one instructor:

CID → Instructor

For example:

201 → Kumar

202 → Ravi

203 → Meena

Therefore:

CID → Instructor

These two dependencies can also be combined as:

CID → CName, Instructor

FD 4: Student ID and Course ID determine Grade

A grade belongs to a particular student-course enrollment.

Therefore, both SID and CID are required:

SID, CID $\rightarrow$ Grade

For example:

(101, 201) $\rightarrow$ A

(101, 202) $\rightarrow$ B

(102, 201) $\rightarrow$ A

Therefore:

SID, CID $\rightarrow$ Grade

3. Complete Set of Important Functional Dependencies

Thus, the main functional dependencies are:

1. SID $\rightarrow$ SName
2. CID $\rightarrow$ CName
3. CID $\rightarrow$ Instructor
4. SID, CID $\rightarrow$ Grade

Or in compact form:

SID $\rightarrow$ SName

CID $\rightarrow$ CName, Instructor

SID, CID $\rightarrow$ Grade

Important Note

There are also derived functional dependencies obtained from the above dependencies.

For example:

SID, CID $\rightarrow$ SName

because:

SID $\rightarrow$ SName

Similarly:

SID, CID $\rightarrow$ CName, Instructor

because:

CID $\rightarrow$ CName, Instructor

These are implied dependencies and do not need to be listed separately as the basic functional dependencies.

4.Types of Functional Dependencies in STUDENT_COURSE

Given:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

The important types are:

1. Trivial Functional Dependency
2. Non-Trivial Functional Dependency
3. Completely Non-Trivial Functional Dependency
4. Partial Functional Dependency
5. Full Functional Dependency
6. Transitive Functional Dependency

a. Trivial Functional Dependency

A functional dependency is trivial when the attributes on the right-hand side are already part of the attributes on the left-hand side.

Example:

$SID, CID \rightarrow SID$

Here, SID is already present on the left side.

Another example:

$SID, CID \rightarrow CID$

Therefore, these are trivial functional dependencies.

In this relation:

$(SID, CID) \rightarrow SID$

$(SID, CID) \rightarrow CID$

These are mathematically valid but are not useful for identifying the candidate key.

b. Non-Trivial Functional Dependency

A functional dependency is non-trivial when the right-hand-side attribute is not a subset of the left-hand-side attributes.

Examples:

$SID \rightarrow SName$

$CID \rightarrow CName$

$CID \rightarrow Instructor$

$SID, CID \rightarrow Grade$

For example:

$SID \rightarrow SName$

SName is not present on the left side.

Therefore, it is a non-trivial functional dependency.

c. Completely Non-Trivial Functional Dependency

A functional dependency is completely non-trivial when there is no common attribute between the left-hand side and right-hand side.

For example:

$SID \rightarrow SName$

Left side:

$\{SID\}$

Right side:

$\{SName\}$

There is no common attribute.

Therefore:

$SID \rightarrow SName$ is completely non-trivial.

Similarly:

$CID \rightarrow CName$

$CID \rightarrow Instructor$

$SID, CID \rightarrow Grade$

are completely non-trivial.

d. Partial Functional Dependency

A partial dependency occurs when a non-key attribute depends on only a part of a composite candidate key.

Our candidate key is:

(SID, CID)

Consider:

$SID \rightarrow SName$

SID is only part of the candidate key (SID, CID) , but it determines $SName$.

Therefore:

$SID \rightarrow SName$ is a partial functional dependency.

Similarly, if course information is considered non-key attributes:

$CID \rightarrow CName$

$CID \rightarrow Instructor$

CID is also only part of (SID, CID) .

Therefore:

$CID \rightarrow CName$

$CID \rightarrow Instructor$

are also partial functional dependencies.

Important conclusion

The relation contains partial dependencies:

$SID \rightarrow SName$

$CID \rightarrow CName$

$CID \rightarrow Instructor$

This means the relation does not satisfy Second Normal Form (2NF) under the stated assumptions.

e. Full Functional Dependency

A functional dependency is full when the entire candidate key is required to determine the attribute on the right side.

Consider:

$SID, CID \rightarrow Grade$

The grade depends on both:

- SID – which student
- CID – which course

If we remove SID :

$CID \rightarrow Grade$

This is not valid because different students can get different grades in the same course.

If we remove CID :

$SID \rightarrow Grade$

This is also not valid because one student can receive different grades in different courses.

Therefore:

$SID, CID \rightarrow Grade$

is a full functional dependency.

f. Transitive Functional Dependency

A transitive dependency occurs when:

$A \rightarrow B$

$B \rightarrow C$

and therefore:

$A \rightarrow C$

For example, if we had:

$SID \rightarrow DepartmentID$

$DepartmentID \rightarrow DepartmentName$

then:

$SID \rightarrow DepartmentName$

would be a transitive dependency.

In the given relation

There is no clear non-trivial transitive dependency among the given attributes under the stated assumptions.

We have:

$SID \rightarrow SName$

$CID \rightarrow CName, Instructor$

$SID, CID \rightarrow Grade$

but there is no dependency such as:

$SID \rightarrow X$

$X \rightarrow SName$

or:

$CID \rightarrow X$

$X \rightarrow CName$

where X is another non-key attribute.

So:

Transitive dependency: Not present in the given relation based on the stated assumptions.

5. Final Functional Dependency Classification

The basic functional dependencies are:

$SID \rightarrow SName$

$CID \rightarrow CName$

$CID \rightarrow Instructor$

$SID, CID \rightarrow Grade$

Their classification is:

$SID \rightarrow SName$

-> Non-trivial

-> Completely non-trivial

-> Partial dependency

$CID \rightarrow CName$

-> Non-trivial

-> Completely non-trivial

-> Partial dependency

CID $\rightarrow$ Instructor

$\rightarrow$ Non-trivial

$\rightarrow$ Completely non-trivial

$\rightarrow$ Partial dependency

SID, CID $\rightarrow$ Grade

$\rightarrow$ Non-trivial

$\rightarrow$ Completely non-trivial

$\rightarrow$ Full dependency

Additionally, trivial dependencies such as:

(SID, CID) $\rightarrow$ SID

(SID, CID) $\rightarrow$ CID

also exist by definition.

Key conclusion:

Candidate Key:

(SID, CID)

Partial dependencies:

SID $\rightarrow$ SName

CID $\rightarrow$ CName

CID $\rightarrow$ Instructor

Full dependency:

SID, CID $\rightarrow$ Grade

Transitive dependency:

Not present based on the given attributes and assumptions.

This classification is useful because it also explains why the original relation is not in 2NF: non-key attributes (SName, CName, and Instructor) depend on only part of the composite candidate key.

6. Finding the Candidate Key

To find a candidate key, we calculate the attribute closure.

Consider:

$K = \{SID, CID\}$

We calculate:

$(SID, CID)^+$

Step 1

Initially:

$$(SID, CID)^+ = \{SID, CID\}$$

Step 2

Using:

$$SID \rightarrow SName$$

we obtain:

$$(SID, CID)^+ = \{SID, CID, SName\}$$

Step 3

Using:

$$CID \rightarrow CName, Instructor$$

we obtain:

$$(SID, CID)^+ = \{SID, CID, SName, CName, Instructor\}$$

Step 4

Using:

$$SID, CID \rightarrow Grade$$

we obtain:

$$(SID, CID)^+ = \{SID, CID, SName, CName, Instructor, Grade\}$$

The closure contains all attributes of the relation:

SID, SName, CID, CName, Instructor, Grade

Therefore:

(SID, CID) is a superkey.

7. Checking Minimality

A candidate key must be a minimal superkey. Therefore, we must check whether either SID or CID can be removed.

Check SID alone

Calculate:

$$SID^+ = \{SID\}$$

Using:

$$SID \rightarrow SName$$

we get:

$$SID^+ = \{SID, SName\}$$

SID cannot determine:

- CID
- CName

- Instructor
- Grade

Therefore:

SID alone is NOT a candidate key.

Check CID alone

Calculate:

$CID^+ = \{CID\}$

Using:

$CID \rightarrow CName, Instructor$

we get:

$CID^+ = \{CID, CName, Instructor\}$

CID cannot determine:

- SID
- SName
- Grade

Therefore:

CID alone is NOT a candidate key.

Check SID and CID together

We already calculated:

$(SID, CID)^+$

$= \{SID, SName, CID, CName, Instructor, Grade\}$

This contains all attributes.

Therefore:

(SID, CID) is a minimal superkey.

Hence:

Candidate Key = (SID, CID)

This demonstrates that:

- The same SID can appear for different courses.
- The same CID can appear for different students.
- The combination of SID and CID is unique.

8. Testing the Candidate Key

```
MySQL 8.0 Command Line Cli  ×  +  ▾

mysql> CREATE TABLE STUDENT_COURSE (
  ->   SID INT,
  ->   SName VARCHAR(50),
  ->   CID INT,
  ->   CName VARCHAR(50),
  ->   Instructor VARCHAR(50),
  ->   Grade CHAR(2),
  ->   PRIMARY KEY (SID, CID)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE DATABASE CollegeDB;
ERROR 1007 (HY000): Can't create database 'collegedb'; database exists
mysql> USE CollegeDB;
Database changed
mysql>
mysql> CREATE TABLE STUDENT_COURSE (
  ->   SID INT,
  ->   SName VARCHAR(50),
  ->   CID INT,
  ->   CName VARCHAR(50),
  ->   Instructor VARCHAR(50),
  ->   Grade CHAR(2),
  ->   PRIMARY KEY (SID, CID)
  -> );
ERROR 1050 (42S01): Table 'student_course' already exists
mysql> INSERT INTO STUDENT_COURSE
  -> VALUES
  -> (101, 'Arun', 201, 'DBMS', 'Kumar', 'A'),
  -> (101, 'Arun', 202, 'Java', 'Ravi', 'B'),
  -> (102, 'Priya', 201, 'DBMS', 'Kumar', 'A'),
  -> (103, 'Rahul', 203, 'Python', 'Meena', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE;
+-----+-----+-----+-----+-----+-----+
| SID | SName | CID | CName | Instructor | Grade |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun  | 201 | DBMS  | Kumar      | A     |
| 101 | Arun  | 202 | Java  | Ravi       | B     |
| 102 | Priya | 201 | DBMS  | Kumar      | A     |
| 103 | Rahul | 203 | Python| Meena      | B     |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> INSERT INTO STUDENT_COURSE
  -> VALUES
  -> (101, 'Arun', 201, 'DBMS', 'Kumar', 'B');
ERROR 1062 (23000): Duplicate entry '101-201' for key 'student_course.PRIMARY'
```

Interpretation:

The STUDENT_COURSE table was successfully created with (SID, CID) as the composite primary key. The insertion of four valid records was successful. The output shows that SID and CID can individually contain duplicate values, but their combination is unique. When an attempt was made to insert a duplicate (SID, CID) combination, MySQL rejected the record with a duplicate primary key error. Therefore, the experiment verifies that (SID, CID) uniquely identifies each student-course record and is the candidate/primary key of the relation.

9. Analysis of Results

From the functional dependency analysis:

- SID determines SName, because each student ID represents one student.
- CID determines CName and Instructor, because each course ID represents one course and its instructor.
- SID and CID together determine Grade, because a grade belongs to a particular student's enrollment in a particular course.
- SID alone cannot identify a complete record.
- CID alone cannot identify a complete record.
- SID and CID together determine every attribute in the relation.
- Therefore, (SID, CID) is the minimal combination that uniquely identifies a record.

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

1. Introduction

First Normal Form (1NF) is the first step in database normalization. A relation is in 1NF when:

- Every attribute contains only atomic (single) values.
- There are no repeating groups or multi-valued attributes.
- Each row represents a single record.
- A key is available to uniquely identify each record.

2. Unnormalized Relation (UNF)

Consider a student-course registration system.

STUDENT_COURSE_UNF

SID	SName	Courses	Instructors	Grades
101	Arun	DBMS, Java	Kumar, Ravi	A, B
102	Priya	DBMS, Python	Kumar, Meena	A, B+
103	Rahul	Python	Meena	B

This relation is unnormalized because some attributes contain multiple values in a single cell.

For example:

SID = 101

Courses = DBMS, Java

Instructors = Kumar, Ravi

Grades = A, B

Therefore, the values are not atomic.

3. Step-by-Step Conversion

Step 1: Identify Repeating Groups

The repeating groups are:

- Courses
- Instructors
- Grades

For SID 101, there are two course registrations:

(DBMS, Kumar, A)

(Java, Ravi, B)

Instead of storing these values in one row, each course registration should be stored as a separate row.

Step 2: Remove Repeating Groups

Separate each repeated value into its own record.

The UNF relation:

```
MySQL 8.0 Command Line Cli  ×  +  v

mysql> CREATE TABLE STUDENT_COURSE_UNF (
  ->   SID INT,
  ->   SName VARCHAR(50),
  ->   Courses VARCHAR(100),
  ->   Instructors VARCHAR(100),
  ->   Grades VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO STUDENT_COURSE_UNF
  -> VALUES
  -> (101, 'Arun', 'DBMS, Java', 'Kumar, Ravi', 'A, B'),
  -> (102, 'Priya', 'DBMS, Python', 'Kumar, Meena', 'A, B+'),
  -> (103, 'Rahul', 'Python', 'Meena', 'B');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE_UNF;
+-----+-----+-----+-----+-----+
| SID | SName | Courses | Instructors | Grades |
+-----+-----+-----+-----+-----+
| 101 | Arun  | DBMS, Java | Kumar, Ravi | A, B   |
| 102 | Priya | DBMS, Python | Kumar, Meena | A, B+  |
| 103 | Rahul | Python   | Meena       | B      |
+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)
```

is converted into

STUDENT_COURSE_1NF:

```
mysql> CREATE TABLE STUDENT_COURSE_1NF (
->     SID INT,
->     SName VARCHAR(50),
->     CID INT,
->     CName VARCHAR(50),
->     Instructor VARCHAR(50),
->     Grade VARCHAR(5),
->     PRIMARY KEY (SID, CID)
-> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO STUDENT_COURSE_1NF
-> VALUES
-> (101, 'Arun', 201, 'DBMS', 'Kumar', 'A'),
-> (101, 'Arun', 202, 'Java', 'Ravi', 'B'),
-> (102, 'Priya', 201, 'DBMS', 'Kumar', 'A'),
-> (102, 'Priya', 203, 'Python', 'Meena', 'B+'),
-> (103, 'Rahul', 203, 'Python', 'Meena', 'B');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE_1NF;
+-----+-----+-----+-----+-----+-----+
| SID | SName | CID | CName | Instructor | Grade |
+-----+-----+-----+-----+-----+-----+
| 101 | Arun  | 201 | DBMS  | Kumar      | A      |
| 101 | Arun  | 202 | Java  | Ravi       | B      |
| 102 | Priya | 201 | DBMS  | Kumar      | A      |
| 102 | Priya | 203 | Python| Meena      | B+     |
| 103 | Rahul | 203 | Python| Meena      | B      |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> |
```

Now, each student-course registration has its own row.

Step 3: Check Atomicity

After conversion, every cell contains only one value.

For example:

Attribute	Value	Atomic
SID	101	Yes
SName	Arun	Yes
CID	201	Yes
CName	DBMS	Yes
Instructor	Kumar	Yes
Grade	A	Yes

There are no values such as:

DBMS, Java

Kumar, Ravi

A, B

inside a single cell.

Therefore, the relation satisfies the atomicity requirement of 1NF.

Step 4: Identify the Key

A student can take more than one course, so:

SID

alone cannot uniquely identify a record.

Similarly, many students can take the same course, so:

CID

alone cannot uniquely identify a record.

The combination:

(SID, CID)

uniquely identifies each student-course enrollment.

Therefore:

Candidate Key = (SID, CID)

For example:

(101, 201)

(101, 202)

(102, 201)

(102, 203)

(103, 203)

are all unique.

Step 5: Verify 1NF Requirements

The converted relation satisfies 1NF because:

1. Atomic Values

Each cell contains only one value.

Satisfied.

2. No Repeating Groups

Multiple courses are stored as separate rows.

Satisfied.

3. Unique Records

(SID, CID) uniquely identifies each record.

Satisfied.

4. Consistent Attributes

Each column contains values of the same type and meaning.

Satisfied.

Therefore, the relation is in First Normal Form (1NF).

4. complete conversion process

The complete conversion process can be represented as:

UNNORMALIZED FORM (UNF)



Identify repeating groups



Separate multiple values



Create one row for each student-course



Ensure atomic values



Identify candidate key (SID, CID)



FIRST NORMAL FORM (1NF)

Conclusion

The unnormalized relation is converted into 1NF by eliminating repeating groups and ensuring that every attribute contains only atomic values. Each student-course enrollment is represented as a separate row.

3. Given relation $R(A,B,C,D,E)$ with FDs: $A \rightarrow BC$, $BC \rightarrow D$, $D \rightarrow E$. Analyze and decompose it to 2NF eliminating partial dependencies.

Given relation:

$R(A, B, C, D, E)$

Functional dependencies:

1. $A \rightarrow BC$
2. $BC \rightarrow D$
3. $D \rightarrow E$

1. Find the Candidate Key

We calculate the closure of A.

A^+

Start with: $A^+ = \{A\}$

Using: $A \rightarrow BC$

we get: $A^+ = \{A, B, C\}$

Using: $BC \rightarrow D$

we get: $A^+ = \{A, B, C, D\}$

Using: $D \rightarrow E$

we get: $A^+ = \{A, B, C, D, E\}$

Therefore:

$A^+ = \{A, B, C, D, E\}$

Since A determines all attributes:

Candidate Key = A

2. Check for Partial Dependency

A partial dependency occurs when a non-key attribute depends on only part of a composite candidate key.

Here, the candidate key is only:

A

It is a single-attribute key, not a composite key.

Therefore, there cannot be any partial dependency.

The given dependencies are:

- $A \rightarrow B, C$
- $BC \rightarrow D$
- $D \rightarrow E$

None of these represents a partial dependency on a part of a composite key.

Important Result

There are no partial dependencies in R.

Therefore, R is already in 2NF.

3. Why R Is Already in 2NF ?

2NF requires:

1. The relation must be in 1NF.
2. There must be no partial dependency of a non-prime attribute on a proper subset of a candidate key.

Since the candidate key is only A, there is no proper subset of the key.

Therefore:

R(A, B, C, D, E) is already in 2NF.

No decomposition is required for 2NF.

4. But There Is a Transitive Dependency

Although the relation is in 2NF, it contains a transitive dependency:

$A \rightarrow BC$

$BC \rightarrow D$

$D \rightarrow E$

Therefore:

$A \rightarrow D \rightarrow E$

So:

$A \rightarrow E$

is a transitive dependency.

This means the relation is not in 3NF.

5. 2NF Result

Since there are no partial dependencies:

R(A, B, C, D, E)

remains unchanged for 2NF.

Dependency Structure

$A \rightarrow B, C$

$BC \rightarrow D$

$D \rightarrow E$

Candidate Key: A

Partial Dependency: None

2NF Decomposition: No decomposition required.

Conclusion

Since A is a single-attribute candidate key, partial dependencies cannot occur. Hence, the given relation R(A,B,C,D,E) is already in 2NF, and no decomposition is required to eliminate partial dependencies.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Given relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

The relation contains a transitive dependency.

1. Identify the Functional Dependencies

The functional dependencies are:

1. EmpID $\rightarrow$ EmpName, DeptID
2. DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptID

and

DeptID $\rightarrow$ DeptName, ManagerID

So, indirectly:

EmpID $\rightarrow$ DeptName, ManagerID

This is a transitive dependency.

2. Identify the Candidate Key

Calculate the closure of EmpID:

EmpID⁺ = {EmpID}

Using:

EmpID $\rightarrow$ EmpName, DeptID

we get:

EmpID⁺ = {EmpID, EmpName, DeptID}

Using:

DeptID $\rightarrow$ DeptName, ManagerID

we get:

EmpID⁺ = {EmpID, EmpName, DeptID, DeptName, ManagerID}

Therefore:

Candidate Key = EmpID

3. Identify the Transitive Dependency

The dependency is:

EmpID $\rightarrow$ DeptID

DeptID $\rightarrow$ DeptName, ManagerID

Therefore:

EmpID $\rightarrow$ DeptName, ManagerID

This is a transitive dependency because DeptName and ManagerID depend on EmpID through DeptID.

4. Decompose into 3NF

To remove the transitive dependency, divide the relation into two relations.

Relation 1: EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

Functional dependency:

EmpID $\rightarrow$ EmpName, DeptID

Primary Key:

EmpID

Relation 2: DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID)

Functional dependency:

DeptID $\rightarrow$ DeptName, ManagerID

Primary Key:

DeptID

5. Final 3NF Relations

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName, ManagerID)

Here:

- EmpID is the Primary Key of EMPLOYEE.
- DeptID is the Primary Key of DEPARTMENT.
- DeptID in EMPLOYEE is a Foreign Key referencing DEPARTMENT.

6. Before and After Decomposition

Before

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID)

EmpID -> DeptID

DeptID -> DeptName, ManagerID

Therefore:

EmpID -> DeptName, ManagerID

This is a transitive dependency.

After 3NF

EMPLOYEE

DEPARTMENT

```
MySQL 8.0 Command Line Cli
mysql> CREATE TABLE DEPARTMENT (
  ->   DeptID VARCHAR(10) PRIMARY KEY,
  ->   DeptName VARCHAR(50),
  ->   ManagerID VARCHAR(10)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE EMPLOYEE (
  ->   EmpID INT PRIMARY KEY,
  ->   EmpName VARCHAR(50),
  ->   DeptID VARCHAR(10),
  ->   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
  -> );
Query OK, 0 rows affected (0.08 sec)

mysql> INSERT INTO DEPARTMENT
  -> VALUES
  -> ('D01', 'IT', 'M01'),
  -> ('D02', 'HR', 'M02');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO EMPLOYEE
  -> VALUES
  -> (101, 'Arun', 'D01'),
  -> (102, 'Priya', 'D02'),
  -> (103, 'Rahul', 'D01');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+
| EmpID | EmpName | DeptID |
+-----+-----+-----+
| 101   | Arun    | D01    |
| 102   | Priya   | D02    |
| 103   | Rahul   | D01    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName | ManagerID |
+-----+-----+-----+
| D01    | IT       | M01       |
| D02    | HR       | M02       |
+-----+-----+-----+
2 rows in set (0.00 sec)
```

Now department information is stored separately and is not repeated for every employee.

Conclusion

The original EMPLOYEE relation contains the transitive dependency $\text{EmpID} \rightarrow \text{DeptID} \rightarrow \text{DeptName}, \text{ManagerID}$. To achieve Third Normal Form (3NF), the department-related attributes are separated into a DEPARTMENT relation. The resulting relations eliminate the transitive dependency and reduce data redundancy and update anomalies.

5. Verify whether $R(A,B,C,D)$ with FDs: $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow A$ is in BCNF. If not, decompose it into BCNF.

Given relation:

$R(A, B, C, D)$

Functional dependencies:

1. **$AB \rightarrow C$**
2. **$C \rightarrow D$**
3. **$D \rightarrow A$**

1. Find the Candidate Keys

Find $(AB)^+$

$(AB)^+ = \{A, B\}$

Using $AB \rightarrow C$:

$(AB)^+ = \{A, B, C\}$

Using $C \rightarrow D$:

$(AB)^+ = \{A, B, C, D\}$

Therefore, AB is a candidate key.

Find $(BC)^+$

Start:

$(BC)^+ = \{B, C\}$

Using $C \rightarrow D$:

$(BC)^+ = \{B, C, D\}$

Using $D \rightarrow A$:

$(BC)^+ = \{A, B, C, D\}$

Therefore, BC is a candidate key.

Find $(BD)^+$

Start:

$(BD)^+ = \{B, D\}$

Using $D \rightarrow A$:

$(BD)^+ = \{A, B, D\}$

Using $AB \rightarrow C$:

$(BD)^+ = \{A, B, C, D\}$

Therefore, BD is also a candidate key.

Candidate Keys

AB, BC, BD

2. Check BCNF Condition

A relation is in BCNF if, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a superkey.

FD 1: $AB \rightarrow C$

AB is a candidate key.

Therefore:

$AB \rightarrow C$ satisfies BCNF.

FD 2: $C \rightarrow D$

Is C a superkey?

Calculate:

$C^+ = \{C, D, A\}$

It does not contain B.

Therefore, C is not a superkey.

So:

$C \rightarrow D$ violates BCNF.

FD 3: $D \rightarrow A$

Calculate:

$D^+ = \{D, A\}$

It does not contain B or C.

Therefore, D is not a superkey.

So:

$D \rightarrow A$ also violates BCNF.

3. Conclusion: Is R in BCNF?

No.

The relation is not in BCNF because:

- $C \rightarrow D$ violates BCNF.
- $D \rightarrow A$ violates BCNF.

4. BCNF Decomposition

We first use the violating dependency:

$C \rightarrow D$

Decompose R into:

R1

R1(C, D)

Functional dependency: $C \rightarrow D$

Here, C is a key of R1, so R1 is in BCNF.

R2

R2(A, B, C)

The remaining attributes are: A, B, C

5. Check R2 for BCNF

In R2:

$AB \rightarrow C$

Since AB determines all attributes of R2:

$(AB)^+ = \{A, B, C\}$

Therefore, AB is a key of R2.

So:

$AB \rightarrow C$ satisfies BCNF.

There are no other non-trivial dependencies that violate BCNF in R2.

Therefore, R2 is in BCNF.

6. Final BCNF Decomposition

The final decomposition is:

R1(C, D)

R2(A, B, C)

R1(C, D)

- FD: $C \rightarrow D$
- Candidate Key: C
- BCNF: Yes

R2(A, B, C)

- FD: $AB \rightarrow C$
- Candidate Key: AB
- BCNF: Yes

Conclusion

The relation $R(A, B, C, D)$ is not in BCNF because the functional dependencies $C \rightarrow D$ and $D \rightarrow A$ have determinants that are not superkeys. Therefore, using the violating dependency $C \rightarrow D$, the relation is decomposed into $R_1(C, D)$ and $R_2(A, B, C)$. Both resulting relations satisfy BCNF.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

1. Given Relation

Consider:

R(A, B, C)

Functional dependency:

$A \rightarrow B$

Decompose R into:

- $R_1(A, B)$
- $R_2(A, C)$

We need to verify whether this decomposition is lossless-join.

2. Create the Chase Table

Create one row for each decomposed relation and one column for every attribute.

Row	A	B	C
$R_1(A, B)$	a	b	c_1
$R_2(A, C)$	a	b_2	c

Here:

- a, b, c are distinguished symbols.
- b_2, c_1 are different symbols.

3. Apply the Functional Dependency

Given:

$A \rightarrow B$

Both rows have the same value a for attribute A.

Therefore, by the FD $A \rightarrow B$, their B values must become equal.

So:

$b = b_2$

The table becomes:

Row	A	B	C
$R_1(A,B)$	a	b	c ₁
$R_2(A,C)$	a	b ₂	c

4. Check the Chase Result

After applying the FD:

$A \rightarrow B$

we obtain:

Row	A	B	C
R_1	a	b	c ₁
R_2	a	b	c

The second row contains:

a, b, c

where all values are distinguished symbols.

Therefore, the decomposition is lossless-join.

5. Why Is It Lossless?

Joining the decomposed relations:

$R_1(A,B) \bowtie R_2(A,C)$

will reconstruct the original relation $R(A,B,C)$ without generating spurious tuples.

Therefore:

$R = R_1 \bowtie R_2$

and the decomposition is lossless.

6. MySQL Experimental Verification

```
mysql> CREATE TABLE R (  
->   A INT,  
->   B INT,  
->   C INT  
-> );  
Query OK, 0 rows affected (0.04 sec)  
  
mysql>  
mysql> INSERT INTO R VALUES  
-> (1, 10, 100),  
-> (2, 20, 200);  
Query OK, 2 rows affected (0.01 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql> CREATE TABLE R1 AS  
-> SELECT A, B FROM R;  
Query OK, 2 rows affected (0.05 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql>  
mysql> CREATE TABLE R2 AS  
-> SELECT A, C FROM R;  
Query OK, 2 rows affected (0.04 sec)  
Records: 2  Duplicates: 0  Warnings: 0  
  
mysql> SELECT R1.A, R1.B, R2.C  
-> FROM R1  
-> JOIN R2 ON R1.A = R2.A;  
+-----+-----+-----+  
| A      | B      | C      |  
+-----+-----+-----+  
| 1      | 10     | 100    |  
| 2      | 20     | 200    |  
+-----+-----+-----+  
2 rows in set (0.01 sec)
```

Conclusion

The tabular Chase algorithm experimentally verifies that the decomposition $R(A,B,C) \rightarrow R_1(A,B)$, $R_2(A,C)$ is lossless, because joining the decomposed relations reconstructs the original relation without spurious tuples.

7. Identify the multi-valued dependencies in **TEACHER(TID, Subject, Hobby)** and decompose it into 4NF.

Given relation:

TEACHER(TID, Subject, Hobby)

Assume a teacher can teach multiple subjects and have multiple hobbies, and these two facts are independent.

1. Multi-Valued Dependencies

The multi-valued dependencies are:

$TID \twoheadrightarrow Subject$

$TID \twoheadrightarrow Hobby$

This means:

- A teacher can have multiple subjects independently.
- A teacher can have multiple hobbies independently.

For example:

TEACHER

TID	Subject	Hobby
T01	DBMS	Cricket
T01	Java	Music
T01	DBMS	Music
T01	Java	Cricket

The combinations create redundancy because subjects and hobbies are independent.

2. Check 4NF

A relation is in 4NF if, for every non-trivial MVD:

$X \twoheadrightarrow Y$

X must be a superkey.

Here:

$TID \twoheadrightarrow Subject$

But TID alone is not a superkey because it does not determine one unique subject and hobby.

Therefore, the relation violates 4NF.

3. Decompose into 4NF

Using:

TID $\rightarrow\rightarrow$ Subject

decompose the relation into:

TEACHER_SUBJECT

TEACHER_SUBJECT(TID, Subject)

```
mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+-----+
| TID | Subject |
+-----+-----+
| 1 | DBMS |
| 1 | Java |
+-----+-----+
2 rows in set (0.00 sec)
```

TEACHER_HOBBY

TEACHER_HOBBY(TID, Hobby)

```
mysql> SELECT * FROM TEACHER_HOBBY;
+-----+-----+
| TID | Hobby |
+-----+-----+
| 1 | Cricket |
| 1 | Music |
+-----+-----+
2 rows in set (0.00 sec)
```

4. MySQL Implementation

```
MySQL 8.0 Command Line Cli X + v

mysql> CREATE TABLE TEACHER (
  ->   TID INT,
  ->   Subject VARCHAR(50),
  ->   Hobby VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> INSERT INTO TEACHER VALUES
  -> (1, 'DBMS', 'Cricket'),
  -> (1, 'Java', 'Music'),
  -> (1, 'DBMS', 'Music'),
  -> (1, 'Java', 'Cricket');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> CREATE TABLE TEACHER_SUBJECT (
  ->   TID INT,
  ->   Subject VARCHAR(50),
  ->   PRIMARY KEY (TID, Subject)
  -> );
Query OK, 0 rows affected (0.06 sec)

mysql>
mysql> CREATE TABLE TEACHER_HOBBY (
  ->   TID INT,
  ->   Hobby VARCHAR(50),
  ->   PRIMARY KEY (TID, Hobby)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER_SUBJECT VALUES
  -> (1, 'DBMS'),
  -> (1, 'Java');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO TEACHER_HOBBY VALUES
  -> (1, 'Cricket'),
  -> (1, 'Music');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+-----+
| TID | Subject |
+-----+-----+
| 1 | DBMS |
| 1 | Java |
+-----+-----+
2 rows in set (0.00 sec)

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+-----+
| TID | Hobby |
+-----+-----+
| 1 | Cricket |
| 1 | Music |
+-----+-----+
2 rows in set (0.00 sec)

mysql> |
```

5. Verify 4NF

TEACHER_SUBJECT:

The relation satisfies 4NF because the multivalued dependency $TID \twoheadrightarrow Subject$ has no independent multivalued attribute remaining in the relation.

TEACHER_HOBBY:

The relation satisfies 4NF because the multivalued dependency $TID \twoheadrightarrow Hobby$ has no independent multivalued attribute remaining in the relation.

Therefore, both TEACHER_SUBJECT and TEACHER_HOBBY are in 4NF.

Conclusion

The relation TEACHER(TID, Subject, Hobby) contains the multi-valued dependencies $TID \twoheadrightarrow Subject$ and $TID \twoheadrightarrow Hobby$. Since TID is not a superkey, the relation violates 4NF. It is decomposed into TEACHER_SUBJECT(TID, Subject) and TEACHER_HOBBY(TID, Hobby), and both resulting relations satisfy 4NF.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

1. Given Relation

Consider the relation:

SUPPLIER_PART_PROJECT(Supplier, Part, Project)

Assume a supplier can supply a part to a project independently of the other relationships.

Example:

SUPPLIER_PART_PROJECT

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Here, storing all three attributes together can create redundancy.

2. Join Dependency

The relation has the join dependency:

(Supplier, Part, Project)

It can be decomposed into:

- SUPPLIER_PART(Supplier, Part)
- SUPPLIER_PROJECT(Supplier, Project)
- PART_PROJECT(Part, Project)

The original relation can be reconstructed by joining these three relations.

3. Redundancy Problem

Supplier	Part	Project
S1	P1	J1
S1	P1	J2
S1	P2	J1
S1	P2	J2

Suppose we have:

Supplier Part Project

The same supplier-part and supplier-project relationships are repeated across multiple rows.

If a new project J3 is added for supplier S1 and parts P1 and P2, several tuples may need to be inserted.

This causes redundancy and modification anomalies.

4. Decomposition into 5NF

```
MySQL 8.0 Command Line Cli
mysql> CREATE TABLE SUPPLIER_PART (
  -> Supplier VARCHAR(10),
  -> Part VARCHAR(10),
  -> PRIMARY KEY (Supplier, Part)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE SUPPLIER_PROJECT (
  -> Supplier VARCHAR(10),
  -> Project VARCHAR(10),
  -> PRIMARY KEY (Supplier, Project)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql>
mysql> CREATE TABLE PART_PROJECT (
  -> Part VARCHAR(10),
  -> Project VARCHAR(10),
  -> PRIMARY KEY (Part, Project)
  -> );
Query OK, 0 rows affected (0.04 sec)

mysql> INSERT INTO SUPPLIER_PART VALUES
  -> ('S1', 'P1'),
  -> ('S1', 'P2');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO SUPPLIER_PROJECT VALUES
  -> ('S1', 'J1'),
  -> ('S1', 'J2');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0

mysql>
mysql> INSERT INTO PART_PROJECT VALUES
  -> ('P1', 'J1'),
  -> ('P1', 'J2'),
  -> ('P2', 'J1'),
  -> ('P2', 'J2');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0

mysql> SELECT SP.Supplier, SP.Part, SJ.Project
  -> FROM SUPPLIER_PART SP
  -> JOIN SUPPLIER_PROJECT SJ
  -> ON SP.Supplier = SJ.Supplier
  -> JOIN PART_PROJECT PP
  -> ON SP.Part = PP.Part
  -> AND SJ.Project = PP.Project;
+-----+-----+-----+
| Supplier | Part | Project |
+-----+-----+-----+
| S1       | P1   | J1      |
| S1       | P1   | J2      |
| S1       | P2   | J1      |
| S1       | P2   | J2      |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> |
```


The original relation:

SUPPLIER_PART_PROJECT(Supplier, Part, Project)

is decomposed into:

SUPPLIER_PART(Supplier, Part)

SUPPLIER_PROJECT(Supplier, Project)

PART_PROJECT(Part, Project)

This decomposition removes join-dependent redundancy and satisfies 5NF (Project-Join Normal Form) when the three binary relationships are sufficient to reconstruct the original relation without spurious tuples.

Conclusion

Join dependencies can cause redundancy when independent relationships are stored together in one relation. Decomposing the relation into smaller relations and reconstructing it through joins eliminates this redundancy and achieves 5NF.

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Consider the following poorly designed college relation:

COLLEGE(EmpID, EmpName, DeptID, DeptName, CourseID, CourseName, Grade)

Sample Data

COLLEGE

EmpID	EmpName	DeptID	DeptName	CourseID	CourseName	Grade
101	Arun	D01	CSE	C01	DBMS	A
101	Arun	D01	CSE	C02	Java	B
102	Priya	D02	ECE	C03	Networks	A
103	Rahul	D01	CSE	C01	DBMS	B

1. Functional Dependencies

The important functional dependencies are:

- $\text{EmpID} \rightarrow \text{EmpName}, \text{DeptID}$
- $\text{DeptID} \rightarrow \text{DeptName}$
- $\text{CourseID} \rightarrow \text{CourseName}$
- $\text{EmpID}, \text{CourseID} \rightarrow \text{Grade}$

Therefore, the candidate key is:

(EmpID, CourseID)

2. Identify the Anomalies

A. Insertion Anomaly

Suppose a new course C04 – Python is introduced, but no student has enrolled in it yet.

We cannot insert the course information without providing an EmpID.

Problem: Course information depends on student enrollment.

B. Update Anomaly

Suppose the department name CSE changes to Computer Science.

The value must be updated in multiple records:

101 $\rightarrow$ CSE

103 $\rightarrow$ CSE

If one record is not updated, inconsistent data will occur.

Problem: The same department information is stored repeatedly.

C. Deletion Anomaly

Suppose student 102 is deleted and Priya was the only student enrolled in course C03.

Deleting Priya's record also removes:

C03 → Networks

D02 → ECE

Thus, course and department information may be lost.

Problem: Deleting student data can unintentionally delete important course/department data.

3. Normalize to 1NF

The relation already contains atomic values, so it satisfies 1NF.

COLLEGE(EmpID, EmpName, DeptID, DeptName, CourseID, CourseName, Grade)

Candidate Key:

(EmpID, CourseID)

4. Convert to 2NF

There are partial dependencies:

EmpID → EmpName, DeptID

CourseID → CourseName

These depend on only part of the composite key (EmpID, CourseID).

Therefore, decompose into:

STUDENT

STUDENT(EmpID, EmpName, DeptID)

COURSE

COURSE(CourseID, CourseName)

ENROLLMENT

ENROLLMENT(EmpID, CourseID, Grade)

5. Convert to 3NF

There is a transitive dependency:

EmpID → DeptID

DeptID → DeptName

Therefore:

EmpID → DeptName

is a transitive dependency.

Separate department information.

STUDENT

STUDENT(EmpID, EmpName, DeptID)

DEPARTMENT

DEPARTMENT(DeptID, DeptName)

COURSE

COURSE(CourseID, CourseName)

ENROLLMENT

ENROLLMENT(EmpID, CourseID, Grade)

6. Final 3NF Schema

STUDENT(EmpID, EmpName, DeptID)

DEPARTMENT(DeptID, DeptName)

COURSE(CourseID, CourseName)

ENROLLMENT(EmpID, CourseID, Grade)

Keys

Relation	Primary Key	Foreign Key
STUDENT	EmpID	DeptID
DEPARTMENT	DeptID	—
COURSE	CourseID	—
ENROLLMENT	(EmpID, CourseID)	EmpID, CourseID

7. MySQL Implementation

```
MySQL 8.0 Command Line Cli × + v

mysql> CREATE TABLE DEPARTMENT (
  ->   DeptID VARCHAR(10) PRIMARY KEY,
  ->   DeptName VARCHAR(50)
  -> );
ERROR 1050 (42S01): Table 'department' already exists
mysql>
mysql> CREATE TABLE STUDENT (
  ->   EmpID INT PRIMARY KEY,
  ->   EmpName VARCHAR(50),
  ->   DeptID VARCHAR(10),
  ->   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
  -> );
Query OK, 0 rows affected (0.15 sec)

mysql>
mysql> CREATE TABLE COURSE (
  ->   CourseID VARCHAR(10) PRIMARY KEY,
  ->   CourseName VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.03 sec)

mysql>
mysql> CREATE TABLE ENROLLMENT (
  ->   EmpID INT,
  ->   CourseID VARCHAR(10),
  ->   Grade VARCHAR(5),
  ->   PRIMARY KEY (EmpID, CourseID),
  ->   FOREIGN KEY (EmpID) REFERENCES STUDENT(EmpID),
  ->   FOREIGN KEY (CourseID) REFERENCES COURSE(CourseID)
  -> );
Query OK, 0 rows affected (0.07 sec)

mysql> INSERT INTO DEPARTMENT VALUES
  -> ('D01', 'CSE'),
  -> ('D02', 'ECE');
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> INSERT INTO STUDENT VALUES
  -> (101, 'Arun', 'D01'),
  -> (102, 'Priya', 'D02'),
  -> (103, 'Rahul', 'D01');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO COURSE VALUES
  -> ('C01', 'DBMS'),
  -> ('C02', 'Java'),
  -> ('C03', 'Networks');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> INSERT INTO ENROLLMENT VALUES
  -> (101, 'C01', 'A'),
  -> (101, 'C02', 'B'),
  -> (102, 'C03', 'A'),
  -> (103, 'C01', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
MySQL 8.0 Command Line Cli × + ∨

mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName | ManagerID |
+-----+-----+-----+
| D01    | IT       | M01       |
| D02    | HR       | M02       |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| EmpID | EmpName | DeptID |
+-----+-----+-----+
| 101   | Arun    | D01    |
| 102   | Priya   | D02    |
| 103   | Rahul   | D01    |
+-----+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM COURSE;
+-----+-----+
| CourseID | CourseName |
+-----+-----+
| C01      | DBMS       |
| C02      | Java       |
| C03      | Networks   |
+-----+-----+
3 rows in set (0.00 sec)

mysql>
mysql> SELECT * FROM ENROLLMENT;
+-----+-----+-----+
| EmpID | CourseID | Grade |
+-----+-----+-----+
| 101   | C01      | A     |
| 101   | C02      | B     |
| 102   | C03      | A     |
| 103   | C01      | B     |
+-----+-----+-----+
4 rows in set (0.00 sec)

mysql> SELECT
  -> S.EmpID,
  -> S.EmpName,
  -> D.DeptName,
  -> C.CourseID,
  -> C.CourseName,
  -> E.Grade
  -> FROM STUDENT S
  -> JOIN DEPARTMENT D
  -> ON S.DeptID = D.DeptID
  -> JOIN ENROLLMENT E
  -> ON S.EmpID = E.EmpID
  -> JOIN COURSE C
  -> ON E.CourseID = C.CourseID;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptName | CourseID | CourseName | Grade |
+-----+-----+-----+-----+-----+-----+
| 101   | Arun    | IT       | C01      | DBMS       | A     |
| 101   | Arun    | IT       | C02      | Java       | B     |
| 103   | Rahul   | IT       | C01      | DBMS       | B     |
| 102   | Priya   | HR       | C03      | Networks   | A     |
+-----+-----+-----+-----+-----+-----+
```

Interpretation

The original relation has been successfully decomposed into DEPARTMENT, STUDENT, COURSE, and ENROLLMENT. The JOIN reconstructs the required information without storing department and course details repeatedly. Thus, insertion, update, and deletion anomalies are reduced, and the resulting schema satisfies 3NF.

Conclusion

The original college database was poorly designed because student, department, course, and enrollment information were stored in a single relation, causing insertion, deletion, and update anomalies. By removing partial dependencies and transitive dependencies, the database is decomposed into four relations in 3NF, reducing redundancy and maintaining data consistency.

10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

To compute the closure of an attribute set, we start with the given attributes and repeatedly apply the functional dependencies (FDs) until no new attributes can be added.

Example

Consider:

R(A, B, C, D, E)

Functional dependencies:

- **$A \rightarrow BC$**
- **$BC \rightarrow D$**
- **$D \rightarrow E$**

1. Compute A^+

Start with:

$$A^+ = \{A\}$$

Using $A \rightarrow BC$:

$$A^+ = \{A, B, C\}$$

Using $BC \rightarrow D$:

$$A^+ = \{A, B, C, D\}$$

Using $D \rightarrow E$:

$$A^+ = \{A, B, C, D, E\}$$

Therefore:

$$A^+ = \{A, B, C, D, E\}$$

Since A^+ contains all attributes of R:

A is a candidate key.

2. Compute BC^+

Start:

$$BC^+ = \{B, C\}$$

Using $BC \rightarrow D$:

$$BC^+ = \{B, C, D\}$$

Using $D \rightarrow E$:

$$BC^+ = \{B, C, D, E\}$$

There is no FD that gives A.

Therefore:

$$BC^+ = \{B, C, D, E\}$$

Since it does not contain A, BC is not a candidate key.

3. Compute D^+

Start:

$$D^+ = \{D\}$$

Using $D \rightarrow E$:

$$D^+ = \{D, E\}$$

Therefore:

$$D^+ = \{D, E\}$$

D is not a candidate key.

4. Candidate Key

The attribute set A determines all attributes:

$$A^+ = \{A, B, C, D, E\}$$

Therefore:

$$\text{Candidate Key} = \{A\}$$

5. Summary

Attribute Set	Closure	Candidate Key
A	$\{A, B, C, D, E\}$	Yes
BC	$\{B, C, D, E\}$	No
D	$\{D, E\}$	No

Final Result

All candidate keys: $\{A\}$

Therefore, A is the only candidate key for the given relation.